CSCC11 - Group 20 Final Report

Ada Cakmak, Kayna Mufidah, Nathanael Kevin Sitepu, Zohair Syed

1. Abstract

This study investigates whether musical attributes can be extrapolated to accurately predict a genre of a song. Using a dataset created by matching track_id's with the Spotify Web API and online sources, we constructed a feature set consisting of numerical audio characteristics, categorical metadata and text-based information. After preprocessing steps such as genre expansion, scaling and encoding we trained and evaluated multiple models including standard KNN, Random Forest, and Logistic Regression along with the more complex Gradient Boosting model. After performing extensive hyperparameter search, we identified that genre prediction is a probable-suited ML problem and that Gradient Boosting achieves the best macro F1-score of about 0.83, demonstrating the effectiveness of ensemble methods for tabular music data.

2. Introduction

Music genre classification is a problem that is prevalent in many music software, but the results can also be expanded for general recommendation systems, playlist generation and overall music organization. The purpose of this study is to improve automatic classification of genres and to understand the possible steps into straying away from manual assignments. The main research question of the study is to understand to what extent these audio features and metadata can be used to accurately predict a song's genre. We also hope to identify if these acoustic features are able to create a separated decision boundary and if certain genres are harder to classify than others. The models in this study will be implemented using the Scikit-Learn Python library and methods range from classical models to more advanced approaches (Pedregosa et al., 2011). For this study, our advanced approach includes a type of ensemble method in Gradient Boosting, which we hope works well with tabular data. We also chose GB models in order to leverage its "additive" nature and account for the hard-to-classify points from the other models. Our initial project proposal included 4 main datasets (Genius lyrics, Billboard Hot-100 data, synthetic Spotify data, and partial Spotify Web API data) but after a failed attempt to merge them with a composite key, we ultimately focused on a unified dataset derived from the Spotify Web API.

3. Data Collection, Preprocessing, Representation

As briefly mentioned previously, our final dataset was created by matching the *track_id*'s present in our partial Spotify Web API data to online sources. Now that we had genuine music data to work with, the first step of preprocessing was expanding the genre label. Since the dataset had genres as a list, we had to expand the *genres* column to create additional rows per genre. We then applied syntax filters to group similar groups of datasets together to classify into 10 categories,

as seen in **Figure 3.1**. In terms of the features, we started by including numerical music characteristics (energy, acousticness, etc), as well as including the boolean *explicit* label as well as popularity metrics. We then one-hot-encoded categorical labels like *album_type*, standardized and scaled numeric features such as *tempo* and *track_duration_ms* using the StandardScaler(), and removed track/album IDs to end up with a final set of features.

To understand our dataset, we first plotted the correlation heatmap, as seen in **Figure 3.2**, across numeric features. Many of the audio features showed high levels of correlation (ex: danceability vs valence, energy vs acousticness), which gave us motivation to continue with model creation. We also wanted to experiment with text columns such as the track_name and album_name, so we opted to group text data into one column and use TF-IDF vectorization. The connotation/denotation of words could hint at their genre, so we wanted to see if models could draw any significance. TF-IDF Vectorization first begins with a sparse matrix of vocabulary words and then assigns each value a TF-IDF score based on the words frequency within the document, over the frequency over the collection of documents.

The full list of features included track_popularity, track_duration_ms, explicit, artist_popularity, artist_followers, album_release_date, album_total_tracks, album_type, danceability, energy, acousticness, instrumentalness, valence, tempo, speechiness, and the artist/song text names.

4. Model Comparison Study

Over the course of this study, we examined KNN, Random Forest, Logistic Regression and an advanced Gradient Boosting model. In terms of an evaluation metric, we preferred the macro F1 score (which balances precision and recall per class), rather than raw accuracy which doesn't account for the imbalanced class previously revealed in **Figure 3.1**. Hyperparameter selection was performed using 5-fold stratified cross-validation on the training set. Stratification ensures each fold has the same class distribution as the full dataset, which is important here since the genre classes are imbalanced. In the final results, we compare each model using the hyperparameters that maximized the chosen metric chosen across all folds.

4.1 K-Nearest Neighbors (KNN)

KNN was selected as a baseline model as it is non-parametric and makes no prior assumptions about the data distribution. Naturally, intuition tells us that songs with similar audio features (tempo, energy, danceability) are likely to belong to the same genre.

The feature set consisted of nine numerical features extracted from the dataset: danceability, energy, acousticness, instrumentalness, valence, tempo, speechiness, track popularity, and track duration. Text columns were excluded as KNN struggles with the high-dimensional representation used with TF-IDF representations.

Three hyperparameters were tuned: *k* (number of neighbours), *metric* (Manhattan vs Euclidean distance), and *weights* (uniform or distance). For *k*, odd values from 1 to *sqrt(len(dataset))* were searched, as larger values begin averaging over too many neighbours and underfitting. We also preferred odd values to avoid ties in majority voting. In terms of *metric* and *weights*, all combinations were tested, and preferences were given to the highest model evaluation.

The optimal model had k=17, metric=manhattan and weights=distance, as shown in **Figure 4.1**.

4.2 Random Forest (RF)

For the second model, Random Forest was selected because it can handle nonlinear relationships between audio features and genre, and it also provides feature importance scores that offer interpretable insight into which audio characteristics relate to genre classification. Importantly, scaling was not applied since scaled features do not help mediate the data and instead only change the thresholds for tree-based models.

Four hyperparameters were tuned: *n_estimators* controlled the number of trees and we tested the values [50, 100, 200, 300], *max_depth* controlled how deep each tree grows and we tried the values [5, 10, 20, None]. We chose the *None* value since it allows trees to grow until leaves are pure or until *min_samples_leaf* is reached. *min_samples_leaf* controls the minimum number of samples required at a leaf node, we tried the values [1, 5, 10, 20]. *max_features* controls how many features are randomly considered at each split. Values tried were [sqrt(F), $\log_2$(F), 0.5], where F represents the number of features in the dataset.

After running a grid search, the optimal hyperparameters chosen were n_estimators=300, max_depth=None, min_samples_leaf=1, max_features=0.5**,** as shown in **Figure 4.2.1**.

4.3 Logistic Regression (LR)

Though it might not be the perfect fit, logistic regression is chosen due to its efficiency and ease of use. The goal of working on this model is to try to discover the existence of a linear boundary that is able to separate the genres.

For the logistic regression models, we chose a wider range of available features, since the model is not restricted and handles both discretes and continuous features. Since extracting text features will produce a sparse matrix and scaling is needed, we will choose the scaler MaxAbsScaler() based on the results of **Figure 4.3.1**.

For the main hyperparameter grid search, we iterate over following:

- *C*, the inverse regularization strength, tested with 20 log-spaced points from 10^{-3} to 10^{3}
- *max_iter*, the iterations until convergence, with values [100, 200, 400, 800, 1000]
- *l1_ratio,* the penalty ratio for L1 and L2, with values [0.0, 0.25, 0.5, 0.75, 1.0]

For the rest of the model hyperparameters, we selected *class_weight* = balanced (which automates each class weight to the inverse of its frequency), *solver* = saga (the only solver that allows the use of *l1_ratio* while defaulting the other values) and *penalty* = elasticnet (as this allows us to test both *l1* and *l2* penalties to find a suitable *l1_ratio*). Using l1_ratio=0.75 and max_iter=1000, we see the optimal model has C=483, as shown in **Figure 4.3.2**.

4.4 Gradient Boosting

Gradient boosting is an ensemble technique in machine learning. For this project, we use the decision tree as the base model (weak model). Unlike an individual decision tree, it builds an ensemble of trees sequentially using gradients to guide the addition of new trees. We choose this model because it is known to perform well on tabular data, which is the dataset that we have for this project. To study the impact of both output representation and objective function, we experiment with three variants of the model: *multi:softmax*, *multi:softprob*, and a custom implementation using focal loss.

- The *multi:softmax* objective gives class prediction by selecting the class with the highest predicted score. We include this model as a baseline that is focused on the final decision.
- The *multi:softprob* objective outputs probability distribution across classes. Though it has the same optimization as *multi:softmax*, it keeps probability distribution information.
- Focal loss addresses the class/genre imbalance in our dataset. For a training example with true class *y* and predicted probability *p* to the correct class, the standard entropy loss is $-\ log(p)$. Focal loss introduce a factor $(1\ -\ p)^{\gamma}$ for $\gamma\ \geq\ 0$, resulting in $-\ (1\ -\ p)^{\gamma} log(p)$. When an instance is correctly classified with high *p*, the term $(1\ -\ p)^{\gamma}$ become small, reducing contribution to total loss. The opposite happens for misclassified/low-confidence instances, which make the gradient update to be dominated by hard classes. This is useful for imbalance datasets. In this project, we use $\gamma\ =\ 2.0$. This is a common choice in literature, as it is strong enough to reduce the contribution of well-classified samples, while preserving gradient signal from difficult examples (Lin et al., 2017).

We use the XGBoost library to implement gradient boosting. We implement hyperparameter tuning for the following hyperparameters only: *max_depth*, which is the maximum depth of the tree, with values [3, 6, 9]; *learning_rate*, which controls step size per boosting round, with values [0.01, 0.1, 0.3]; *n_estimators*, which controls the number of trees trained, with values [50, 100, 200]. We choose these values as they are close to the recommended/default hyperparameter value in XGBoost documentation (Chen et al., 2026). We fix some hyperparameters: *subsample*: 0.8, which means we use 80% of training data per tree (we choose this value because the standard recommended range is 0.5 – 1.0, 0.8 gives good balance); *colsample_bytree*: 0.8, which is the feature usage per tree, same reasoning as *subsample*; *num_class*: 12, total classes we predict; *seed*: 1, controls the randomness of the model to ensure reproducibility; and *verbosity*: 0, which suppresses logs. We perform the hyperparameter tuning using 5-fold cross-validation to

maximize the macro F1-score. The tuned hyperparameters are the same across all models: 6 for *max_depth*, 0.1 for *learning_rate*, and 200 for *n_estimators* (**Table 4.4.1**). The model performance using tuned hyperparameters can be seen in **Figure 4.4.2**.

5. Main Results

Using the optimal hyperparameters for each model, we have aggregated the main results into **Tables 5.1 - 5.4**. When using the macro F1 score as the evaluation metric (the harmonic mean of precision and recall per class), we can see the model performances vary from 0.62 (KNN), 0.71 (RF) and 0.68 (LR). We expected LR to potentially outclass the others, especially due to our extensive grid search over the set of hyperparameters, but that was not the case. From **Figure 4.3.2**, it can be seen that the scores obtained by LR increases as C increases up to a certain point. After that point, the precision plateaus. This is consistent with the complexity of logistic regression models. As C increases, the penalty decreases, which allows the model to be more complex. However, logistic regression is still fundamentally a model that finds a linear boundary. Hence, after a certain value of C, the model has found the best boundary and increasing the value any further will not improve the score.

In the end, the advanced Gradient Boosting outclassed classical models, after reaching a top macro F1 score of 0.84. Overall, the results of this study show that genre classification is possible and with 10 labeled bins we can successfully get high evaluations using complex models. However, the challenging nature of this problem stems from the large number of labels generated and class imbalance, evidenced by the ~10% gap between accuracy and balanced accuracy when using the GM model seen in **Table 5.4**. Additionally the results suggest that genres like Latin (0.97 F1-score) may just be more musically discernible than others (ex: Electronic with a 0.62 F1-Score), despite the similar sample size (**Table 4.2.2**). Ultimately, the different models each tried to optimize their decision boundaries and ultimately the softmax GM model outperformed the Focal Loss GM model and showed that the standard cross-entropy was sufficient in capturing the majority of the underlying patterns without over-focusing on noise or severely underfitting the complex task of grouping the songs into the distinct genres.

6. Future Work

Overall, improvements to the current project include more extensive hyperparameter tuning and experimenting with alternative objective functions to optimize each model. We can also explore more advanced (tabular) models such as TabTransformer, TabNet, and others. There might also be gains from further feature engineering: analyzing feature importance, finding correlations between features, creating new features, or incorporating richer data sources. Applying NLP techniques for language-based features could also enhance model performance. Lastly, class imbalance handling, broader evaluation metrics, and more rigorous hyperparameter tuning might also improve the accuracy in the ever-changing problem of categorizing music tracks.

7. Appendix

Figure 3.1: Genre Distribution

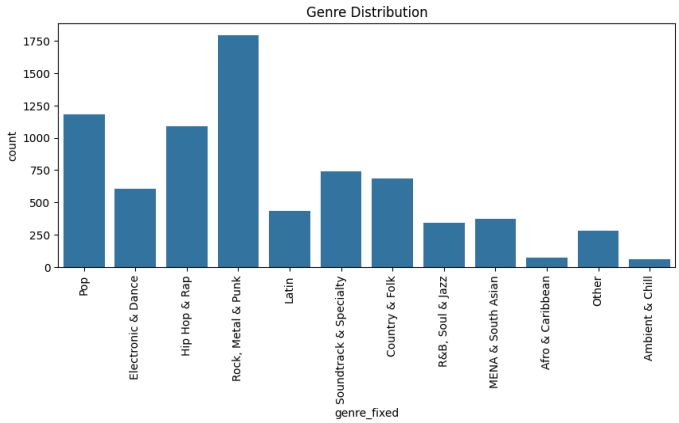

Figure 3.2: Features Correlation Heatmap

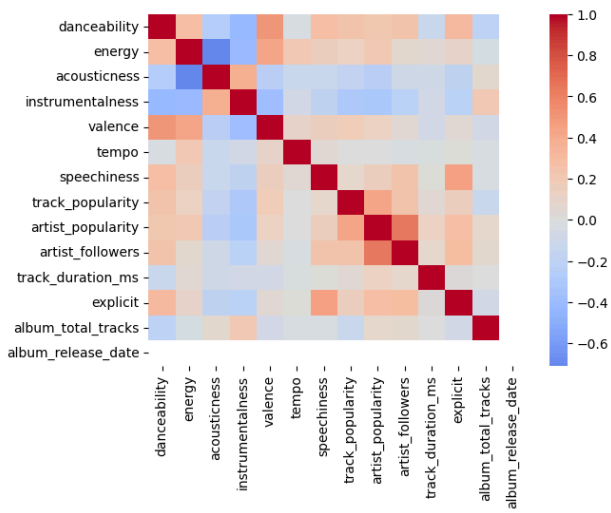

Figure 4.1: KNN Hyperparameter Tuning

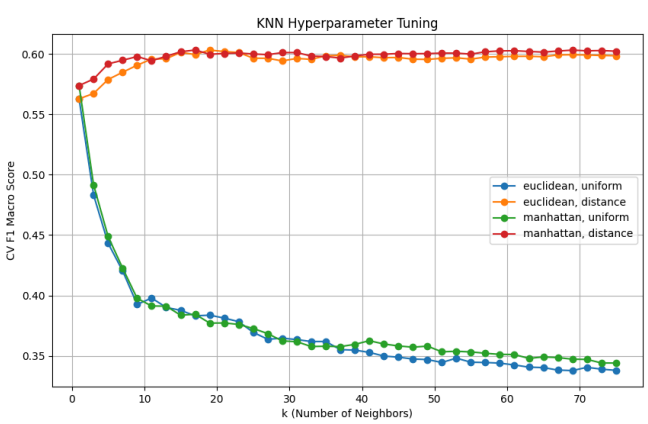

Figure 4.2: RF Hyperparameter Tuning

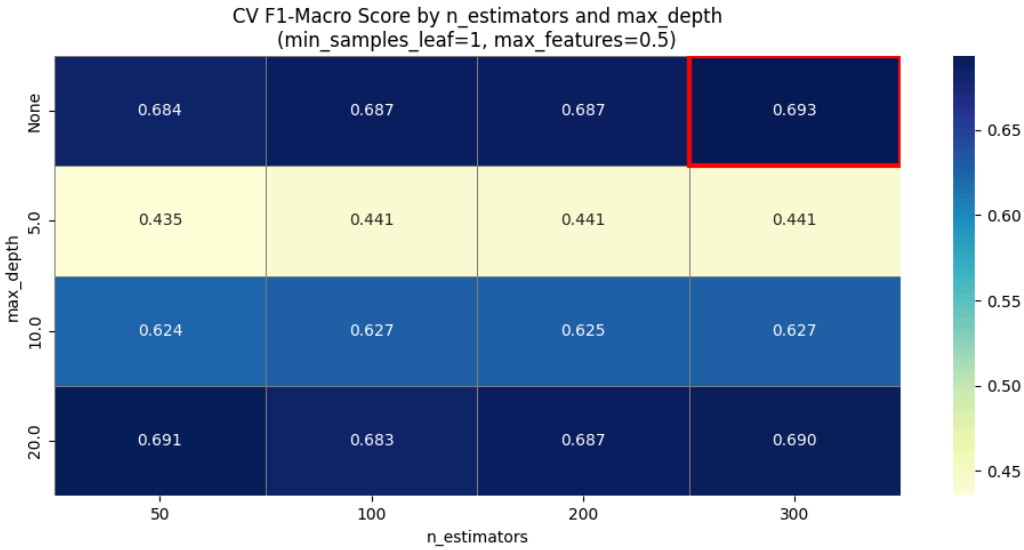

Figure 4.2.2: Random Forest Classification Report by Genre

	precision	recall	f1-score	support
Afro & Caribbean	0.50	0.70	0.58	10
Ambient & Chill	0.46	0.46	0.46	13
Country & Folk	0.73	0.83	0.77	138
Electronic & Dance	0.56	0.70	0.62	91
Hip Hop & Rap	0.92	0.88	0.90	207
Latin	0.97	0.96	0.97	76
MENA & South Asian	0.96	0.96	0.96	70
Other	0.24	0.15	0.18	48
Pop	0.86	0.73	0.79	222
R&B, Soul & Jazz	0.73	0.55	0.62	73
Rock, Metal & Punk	0.85	0.91	0.88	368
Soundtrack & Specialty	0.80	0.84	0.82	137

Figure 4.3.1 Logistic Regression Precision vs Scaling Function

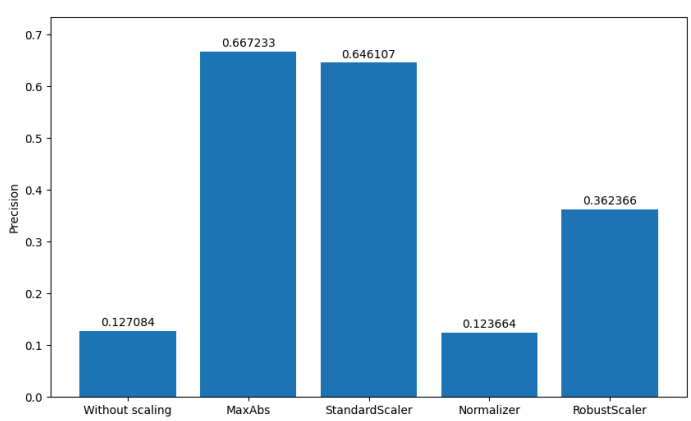

Figure 4.3.2: Precision Plot of the Logistic Regression Model

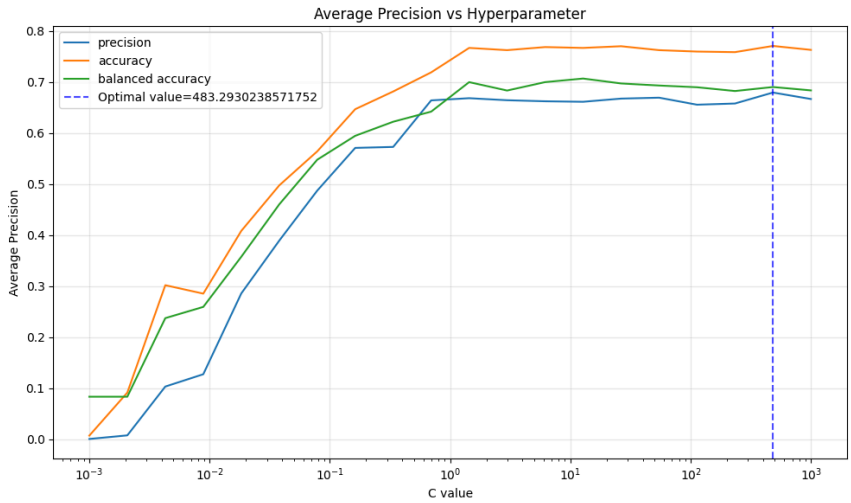

Table 4.4.1: Gradient Boosting hyperparameter result from cross-validation

Model	***max_depth***	***learning_rate***	***n_estimators***	**CV F1 Macro**
Softprob Gradient Boost	6	0.1	200	0.8516
Softmax Gradient Boost	6	0.1	200	0.8496
Focal Loss Gradient Boost	6	0.1	200	0.8447

Figure 4.4.2 Bar Chart for Gradient Boosting Evaluation

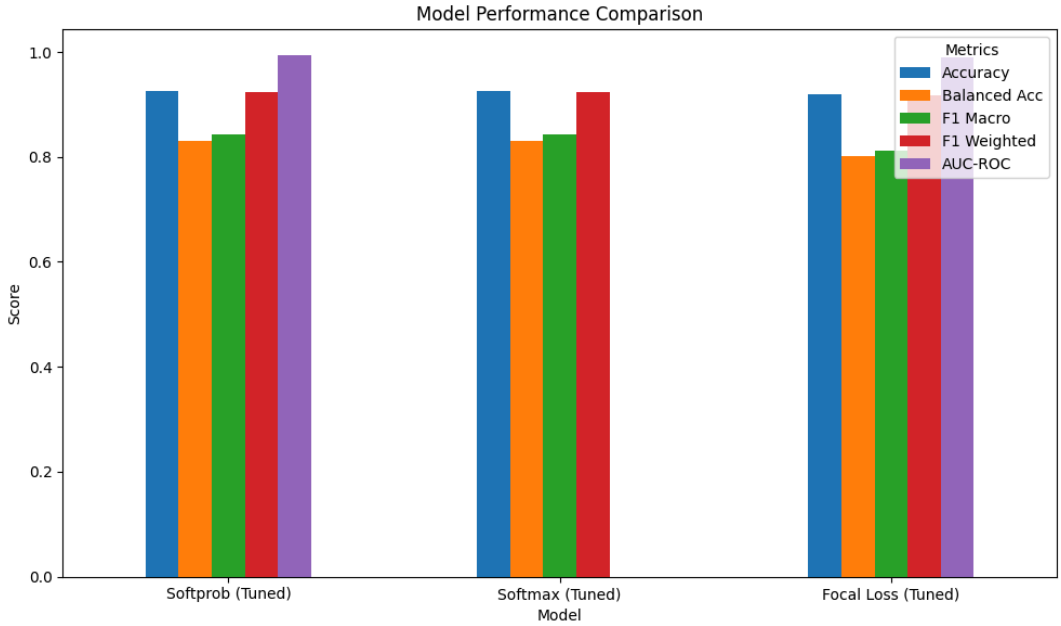

Table 5.1: KNN best hyperparameter results from cross-validation

Metric	**Precision**	**Recall**	**F1-Score**	**Support**
Accuracy	-	-	0.71	1453
Macro Avg	0.62	0.64	0.62	1453
Weighted Avg	0.70	0.71	0.70	1453

Table 5.2: RF best hyperparameter results from cross-validation

Metric	**Precision**	**Recall**	**F1-Score**	**Support**
Accuracy	-	-	0.81	1453
Macro Avg	0.71	0.72	0.71	1453
Weighted Avg	0.80	0.81	0.80	1453

Table 5.3 Logistic Regression best hyperparameter results from cross-validation

Metric	**Precision**	**Recall**	**F1-Score**	**Support**
Accuracy	-	-	0.77	1816
Macro Avg	0.68	0.69	0.68	1816
Weighted Avg	0.78	0.77	0.78	1816

Table 5.4: Evaluation result for Gradient Boosting

Model	**Accuracy**	**Balanced Acc**	**F1 Macro**	**F1 Weighted**	**AUC-ROC**
Softprob	0.9256	0.8289	0.8423	0.9235	0.9931
Softmax	0.9256	0.8298	0.8423	0.9235	N/A
Focal Loss	0.9191	0.8018	0.8128	0.9170	0.9894

8. References

Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, Li M, Xie J, Lin M, Geng Y, Li Y, Yuan J, Cortes D (2026). *xgboost: Extreme Gradient Boosting*. R package version 3.3.0.0, https://github.com/dmlc/xgboost.

Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision* (pp. 2980-2988).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.